作品编号：[系统编号待填写]

作品类别： □软件设计 □硬件制作 ■工程实践

□密码应用技术 □其它

2026年第十一届全国密码技术竞赛作品设计报告

作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护（暂定题名）

2026 年 10 月 7 日

中国密码学会

基本信息表
编号：[系统编号待填写]
作品题目：ML-KEM 解封装实现的时间侧信道检测与常数时间防护（暂定题名）
作品类别：□软件设计 □硬件制作 ■工程实践 □密码技术应用 □其它
作品内容摘要： 本作品构建 C11 本地审计与复算工程，接入固定 ML-KEM-768 便携 C 实现，用独立 NIST 向量、接口边界及版本等价验证功能。普通采集在完整解封装窗口内计时，保留逐次记录、公开输入流摘要、源码与产物身份。历史人工分支回归关联源码、机器码、名义统计与掩码对照，明确其教学审计边界。在真实 4vCPU KVM/x86_64 Linux 取得普通参考及三种编码无分支形态的实现的性能数据；在 8 个固定无秘密任务上，C 与 SciPy 精确复算结果一致，完整进程墙钟比值中位数 0.001389。作品提供统一入口、离线展示与干净复算。新受控防护时序闭环及非原作者执行仍缺，不把未检出、功能通过或复算更快解释为安全认证。
关键词（五个）：ML-KEM；时间侧信道；常数时间；证据资格；可复现性

1.作品功能与性能说明

本作品处理 ML-KEM 软件解封装审计中常见的证据脱节：只有时间差而缺少对象身份，只有源码改造而缺少实际产物，或者把未检出与安全认证混为一谈工程将功能、测量、分析、定位、防护和复现分为可核验步骤，使使用者能够判断一份结果支持到哪一层。

可运行对象固定为 ML-KEM-768；核心算法封装、随机源、输入生成、计时、统计与运行校验为 C11。CMake/CTest 是唯一构建测试来源。audit 入口提供环境及源身份、功能检查、普通采集、描述分析、条件化成对分析、历史复算和离线 HTML 导出。没有远程服务或密钥恢复功能。

当前成果包括独立功能证据、真实 x86_64 Linux 普通记录、历史人工机制防御案例及有限精确复算效率增量。正式批次比较在准备失败处停止，未执行 K4/K16 开发或 20 对确认。工程可用与竞赛所有要求达标是不同状态；后续章节保留尚未闭合的要求。

2.设计与实现方案

2.1 实现原理

ML-KEM 库采用 mlkem-native 便携 C 后端的固定快照，完整提交见来源 [2]，标准口径为 2024 年 FIPS 203。公钥 1184 字节、私钥 2400 字节、密文 1088 字节、共享密钥 32 字节。上游源码和许可证原样保留；向量正确不代表 FIPS 模块认证。[1][2]

封装层首先检查指针和精确长度，避免短缓冲区进入固定长度内核。正确长度的无效密文继续走隐式拒绝，不向 CLI 暴露重加密验证标志。系统随机源失败向调用方传播；公开种子的确定性入口限实验夹具，不用于保护业务密钥。

普通输入为 valid、bit-flipped、random 三类，比例 1:1:1，全部长度 1088 字节。标签来自生成规则，不来自内部接受标志。输入生成、分配、结果消费

及I/O位于计时窗口外，窗口内覆盖完整包装层与内核解封装。数据保存原始顺序，公开种子固定类别乱序，不删除普通长尾。

程序记录时钟、单位、读钟及空窗自检、运行完成声明、实际可执行文件SHA3-256、源身份和编译条件。统一入口仅在数据结构和完成声明通过后导出描述页面；HTML完成写入后独占发布，失败产物保留为部分输出。默认使用归档机制案例，不构建旧主动机制计时入口。

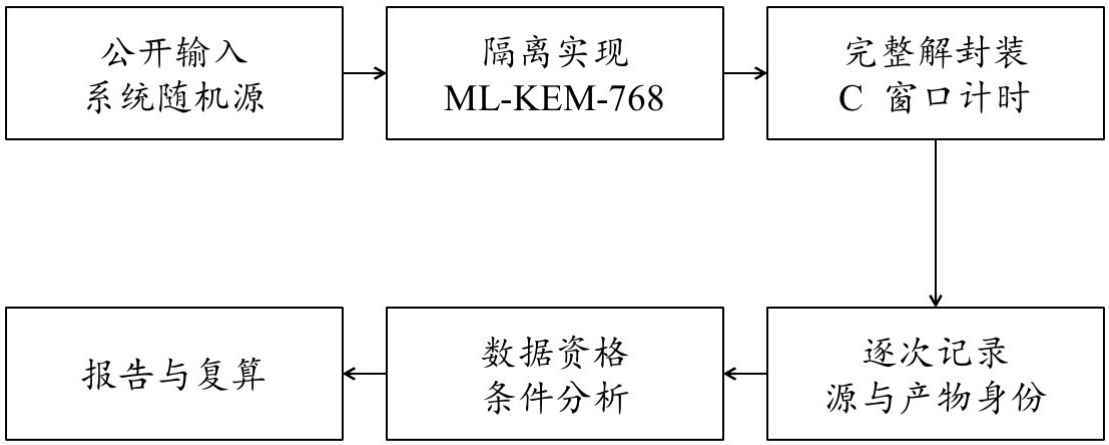


图1 软件流程。实时普通记录、历史重分析和合成统计分别留证。

2.2 运行结果

原生目标环境为Ubuntu 24.04.4、Linux 6.8.0-138、x86_64/KVM、4个vCPU，CPU型号字符串为AMD EPYC 9K65 192-Core Processor。192-Core是型号字符串，不是实例资源。正式普通产物为GCC 13.3.0、RelWithDebInfo、-O2 -g -DNDEBUG、无LTO及非sanitizer，外部taskset绑定CPU2。宿主频率、物理资源和调度无法由来宾控制。

前置参考三次各3000条，共9000条完整记录，均值分别47075.037、46237.526、46121.981ns。CLOCK_MONOTONIC报告分辨率1ns，观察最小正读钟增量19ns；表示单位和报告分辨率不能替代实际精度。每次保存10000个读钟增量及1000个空窗值，未扣开销。源、产物与数据见证据E01。

统一入口已完成参考短采集并导出真实页面；短演示仅证明流程，不替代正式样本。历史 20 份五种子四版本数据生成 40 份报告，与归档逐字一致，未重新采集。功能、历史重分析及实时普通记录分别展示，避免把封存材料伪装成现场实验。

2.3 技术指标

接口以 1088 字节正确长度密文作为普通测量负载；每次固定 100 次预热。逐次 CSV 含运行、实现、类别、序号、纳秒值和单位；配套元数据保留源与产物身份。描述分位数按 floor(p×(n−1)+0.5) 零基下标，不插值。旧报告“nearest rank”名称不准确，但数值和历史兼容输出不改。

新的条件化分析限 4—20 个成对单位、相同调用数和有界整数时间差。令 d_k 为对内总耗时差，T 为差值和的绝对值，枚举全部 2^m 种标签交换，上尾包含等值，alpha 固定 0.01。独立对内交换及无后效条件不足时输出证据不足；效应区间另需成对单位 iid。程序资格声明不是外部独立审计。

这是既有精确随机化方法的工程实现，并未首次提出。C11 利用相邻交换状态更新整数和，避免浮点近似移动等值边界；独立手算及 SciPy 核对验证实现。真实参考后赋独立标签属于无秘密零效应控制，合成偏移属于统计夹具，不能写成密码泄露。

3.系统测试与结果

3.1 测试方案

测试分为功能与接口、记录与来源、统计正确性、失败传播、产物核对及作者复算。独立 NIST 向量提供预期答案，不由被测实现自产。源码和编译条件改变后，对受影响产物重建并执行必要检查；动态检查与正式测量分目录、分时执行。

正常构建覆盖四版 KAT、随机源失败、功能等价、CLI、向量及摘要、记录与数据资格、配对数学和预算失败、统计手算、工作流及源码导出。普通性能固

定 S02 执行 31 项；后续只读复算 S03 增加到 32 项。有限模型校准 S04 新增卷积手算检查，原生固定源码的 GCC 普通与 Clang 动态各 33 项通过。各快照按其实际注册与执行解释，旧主动机制计时不在默认交付范围。

参数、方法、成本边界与有限数据集合在观察确认结果前冻结。开发和确认分开；原始负结果、失败准备和未闭合成本保留，不换种子、改阈值或重开预算得到通过。软件夹具时间不进入真实平台性能结果。

3.2 功能测试

每版独立 KAT 覆盖 25 组 KeyGen、25 组 Encaps、10 组 Decaps 及 20 组密钥校验，解封装包含 5 组隐式拒绝。四版共 320 组独立用例；接口检查包括空指针及错误长度。三组对参考版的差分各覆盖 64 个公开种子和三类输入，拒绝样本另核对 J(z∥c)。

Apple M5/ARM64 实现源的普通及 ASan/UBSan 检查通过。Linux 普通测量固定 S02 在 GCC 及 Clang 动态各执行 31 项，后续只读验收 32 项；有限模型校准固定 S04 两种构建各 33 项。此前 GCC sanitizer 在既有人工补丁的符号转换警告处失败，原日志保留，没有关闭警告或修改封存补丁。动态检查不冒充正式性能产物。

统计正确性包含 4 个相同差的精确尾计数 2/16、零总和 16/16、全并列、整数边界、畸形数据及计划拒绝、覆盖拒绝和依赖资格不足。源码导出按清单验证，不含.git；构建身份明确为导出摘要，兼容字段不冒称 Git 提交。

3.3 性能测试

普通性能在同一 Linux 目标、固定源码 S02 和 6 个运行块执行。每块三种编码无分支形态的实现分别 3000 正式及 100 预热，六种方法排列全部出现；三种实现共享该块公开种子、有序输入流和负载比例。18 个完整运行，共 54000 条正式记录、1800 次预热；采集过程中不并行安装、编译或其他测量。

逐次数据支持单次 P50/P90/P99；每运行均值独立保留，不池化成同样数量的独立任务。相对参考，hardened 均值差 101.959ns、相对开销

0.220522%，探索性 99%成对 bootstrap 区间−3.725 至 189.399ns；deployable 差 − 30.101ns、开销 − 0.065103%，区间 − 240.918 至 193.997ns。区间均跨零。

6 个运行对的 iid 条件在 KVM 上未得到证明，bootstrap 区间仅是该模型下的探索描述，不保证总体覆盖，也不作防护效果判定。hardened 保留函数级 O0 控制，reference/deployable 正常优化，比较同时包含优化和代码形态差异；不能把速度差完全归于安全防护。负开销允许存在，不能据此声称更安全。

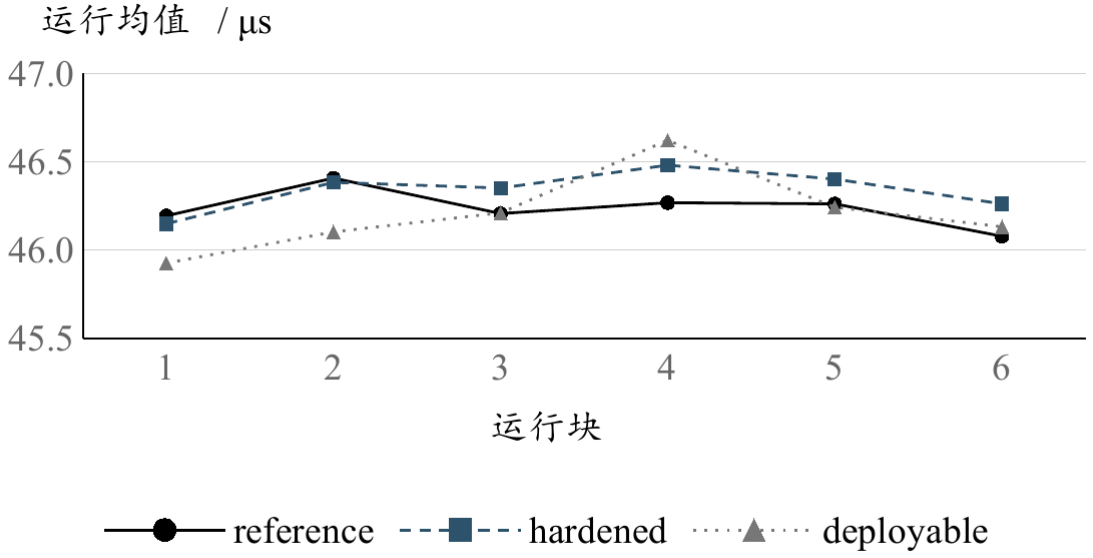


图 2 全部 18 个普通运行均值。单位换算为微秒；非单次记录或总体置信区间，长尾及分位数见表 1 和原始数据。

表 1 原生普通运行的逐次分位数 单位 ns 每运行 3000 条

运行块	实现	均值	P50	P90	P99
1	reference	46193.517	45720	45949	54699
1	hardened	46148.423	45908	46058	50968
1	deployable	45925.623	45688	45829	50667
2	reference	46407.084	45538	48678	60098
2	hardened	46383.034	45849	46129	56419
2	deployable	46102.585	45578	45808	56459
3	reference	46206.193	45688	45929	55897
3	hardened	46350.749	45808	46029	56530

运行块	实现	均值	P50	P90	P99
3	deployable	46209.969	45689	45909	56238
4	reference	46267.377	45729	45950	56348
4	hardened	46480.338	45929	46169	56799
4	deployable	46621.075	45779	47489	60619
5	reference	46261.240	45719	45950	56279
5	hardened	46401.231	45849	46089	56408
5	deployable	46241.867	45718	45938	56498
6	reference	46077.670	45540	45879	56389
6	hardened	46261.061	45719	45949	56549
6	deployable	46131.357	45579	45789	56479

表 1 来源：E02/performance.report 及同名 CSV/meta；原始长尾完整保留。区间见正文，属于运行级条件描述。

3.4 测试数据与结果

历史人工回归在重加密所用消息编码中引入秘密相关分支。消息来自私钥参与的解密，源依赖与实际编译产物可以对应。hardened 在相同优化控制下改用掩码，保持密码语义；deployable 是正常优化下的编码形态对照。上游目标函数本来无分支，二者不是对上游新漏洞的修复。

既有 ARM64 五种子四版本档案中，H1 全零对全一主比较在预定 1e−4 阈值仅 4/5 显著；H2 恒定组对变化组在 1e−3 阈值，人工回归 5/5 显著、reference/deployable 各 5/5 未检出、hardened 有 1/5 显著残差。两轮假设和阈值不同；固定消息组仅两个密文反复测量，名义 p 值不校正序列依赖。既有强措辞以统计审计限制为准。

有限批次路线首次原生准备在 GCC 结构体初始化编译错误处停止，保留 40000 共同调用/3600 秒未闭合预留。K4/K16 开发、选择及 20 对确认未执行，没有稳定性收益或成本比值。后续可移植性修复和普通验收不能回填成该合同成功。

统计主方向尚无超越正确配置 dudect 或 RTLF 的依据。dudect 是自适应 Welch 检验族，RTLF 理论涉及 iid 及有限方差，不能把既有相依记录误用造成的失败当胜利。[3][4] 备用方向采用 SciPy1.16.3/NumPy2.3.5 的精确成对交换、向量化、batch4096，与 C 使用相同 20 对整数数据、完整 2^20 分母、上尾含等值及同一检验统计量。[5]

开发 2 份固定文件、12 次进程只用于兼容与选择；确认采用 8 份新冻结无秘密任务，每方法每份 3 次，共 48 次完整进程，精确尾计数及判定全部一致，原始 CSV 未变。每份最大 C 耗时均小于最小 SciPy 耗时，24 个比值中位数 0.001388504，范围 0.001139408—0.001734854。SciPy 总进程 56.401718 秒，C 为 0.080189 秒；确认观察窗口 56.487808 秒，开发 14.048571 秒，均在固定总预算内。

这支持限定任务的完整进程复算效率增量，启动、读取、统计和输出均计入；Python 导入也计入基线。它不说明密码解封装加速、纯计算内核速度、误报率、检出率或密码安全性提高。8 个固定任务不代表密码总体，未给总体速度置信区间。此前接口验证和研发完整墙钟未采集，不宣称全研发成本闭合。

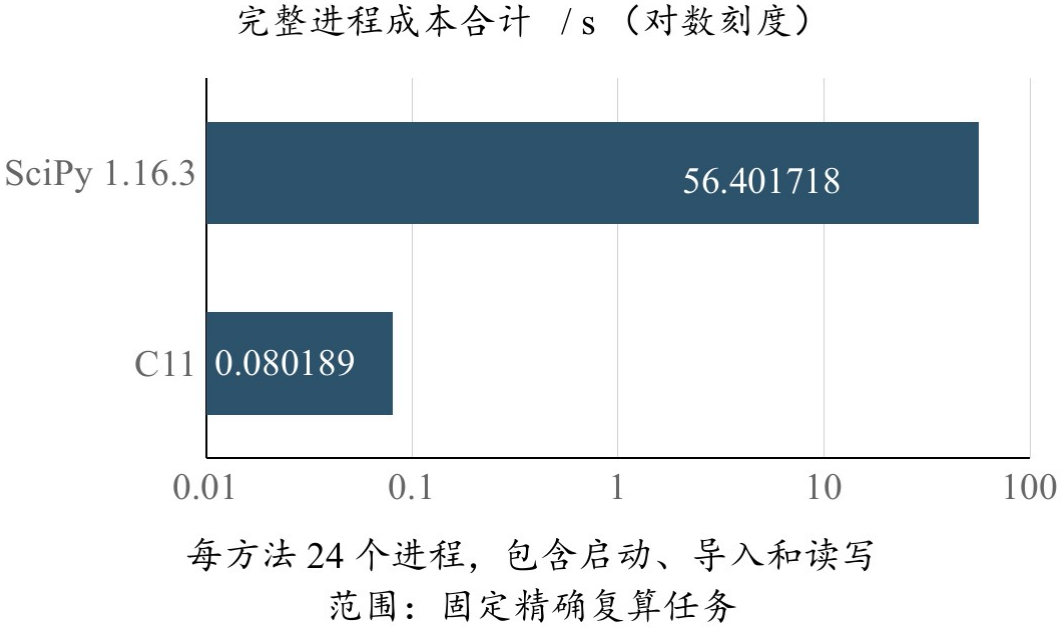


图 3 每种方法 24 个完整进程的成本合计，采用对数刻度。源为 C 观察报告；导入与启动计入 SciPy，不作纯内核性能或密码安全声明。

表 2 有限效率确认　完整精确检验判定逐项一致

项	结果
固定确认任务	8 份新冻结文件，各 20 对
实际完整进程	48 次，退出码全部 0
比值中位数	0.001388504
比值范围	0.001139408—0.001734854
原始身份	CSV 及冻结计划未修改
支持范围	限定完整进程复算效率

3.5 有限模型统计校准

在观察结果前固定 12 个成对单位。equal 每对幅度为 1，graded 依次为 1 至 12；差值为 delta 加正负幅度，五个效应值见表 3。独立模型的全部 4096 种符号组合等概率；共同符号模型各仅两种结果。共 40964 份数据由 C 生成和验证，单位为合成 ns，不是密码采集时间；没有新的 ML-KEM 解封装调用，不把模型当真实密码噪声分布。

每份数据分别用现有精确入口和另行实现的整数分布卷积核对，40964 份尾计数全部一致；44 份预先选定的 SciPy 审计也一致。独立零效应的误拒绝概率为 26/4096 和 38/4096，均低于 alpha=0.01；equal 的结果与二项式手算吻合。表 3 保留全部效应结果，graded 在效应 4 时仍只有 353/4096 检出；equal 在效应 1 时的一份全零数据保留在分母中。

共同符号模型虽然均值为零，盲用独立交换规则时每个模型都是 2/2 拒绝；实际成对入口对四份依赖数据均返回证据不足。拒绝签收不等于把误报降至零。该补证支持固定有限模型的错误、检出和资格边界，未校准真实密码总体或真实序列依赖，不提供超越正确配置 dudect/RTLF 的检测证据。

表 3 固定有限模型的原始拒绝数　每个独立模型 4096 种结果

效应 delta（合成 ns）	equal 拒绝数	graded 拒绝数
0	26	38
1	794	48
2	4096	91

效应 delta（合成 ns）	equal 拒绝数	graded 拒绝数
4	4096	353
12	4096	4096

表 3 来源：C02 有限模型原始记录及 C 报告；效应 0 为模型内误拒绝，其余为该模型检出，不估计真实密码总体概率。

4.应用前景

作品可作为本地密码实现审计教学及工程复核工具。开发者可以先确认功能和接口，再保存普通运行的完整记录；使用者可从指定文件重分析，核对方法、源身份和限制，减少“只有一个 p 值”或“只看源码分支”的错误判断。
适用范围是单机 ML-KEM-768 便携 C 实现、本地运行与离线复算。未来需要团队在批准的防御测试计划下取得新的受控防护前后数据、进行必要的因果分离，并由实际未参与实现的成员独立执行。云宿主控制不足、其他处理器、优化条件或网络环境的变化都需要新的身份和证据，不能用现有数字外推。

5. 结论

作品已形成可运行的 C11 工程、统一使用入口、真实原生普通测量、历史防御案例档案、无秘密精确统计验证、限定强对照效率结果及原始数据复算材料功能正确、记录完整、有限复算效率和密码安全是四类不同结论。
演示的实际界面截图因工具本地文件访问策略受限，未取得；离线页面和真实程序日志已提供，截图需团队手工完成。未闭合项仍包括新受控防护时序闭环历史差异的完整因果分离、运行独立性校准和真实非原作者执行。新平台普通性能与静态产物核验不能替代这些证据。作品编号、正式题名、实际分工、签字、意见和盖章由团队真实填写及完成；本报告不宣称已经正式提交或全部要求达标。

6 来源与复现材料

[1] NIST. Module-Lattice-Based Key-Encapsulation Mechanism Standard. FIPS 203, 2024. https://csrc.nist.gov/pubs/fips/203/final
[2] pq-code-package. mlkem-native. 固定 b3ba7b32773e657dd37f6f87bce82528459ad8a4；许可证见源码原件。https://github.com/pq-code-package/mlkem-native
[3] Reparaz O, Balasch J, Verbauwhede I. Dude, is my code constant time? DATE 2017；dudect 固定 dc269651fb2567e46755cfb2a13d3875592968b5。https://github.com/oreparaz/dudect
[4] Dunsche M, et al. With Great Power Come Great Side Channels: Statistical Timing Side-Channel Analyses with Bounded Type-1 Errors. USENIX Security 2024；已核对官方勘误。https://www.usenix.org/conference/usenixsecurity24/presentation/dunsche
[5] SciPy. permutation_test 官方定义；对照实际版本 SciPy1.16.3/NumPy2.3.5。https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.permutation_test.html
[6] clangover 机制出处固定 7f4d5dc162b77c362a34a0d52949f7a3e1b16d81，原对象是旧 Kyber，不能冒称 FIPS 203。https://github.com/antoonpurnal/clangover

证据：E01 参考前置，E02 普通性能，H01 历史审计，C01 复算效率，C02 有限模型校准，R01 作者复现。原 CSV 字节不变，元数据角色转换另有摘要。重建命令见支撑包 source/README.md，非原作者须保留实际日志并本人签收。